

LoveCS

The program in *LoveCS.java* prints "I love Computer Science!!" 10 times. Copy it to your directory and compile and run it to see how it works. Then modify it as follows:

```
//
*****
// LoveCS.java // // Use a while loop to print many messages
declaring your // passion for computer science //
*****
public class LoveCS {
public static void main(String[] args)
{
    final int LIMIT = 10;
    int count = 1;
    while (count <= LIMIT)
    {
        System.out.println("I        love        Computer
        Science!!");
        count++;
    }
}
}
```

1. Instead of using constant LIMIT, ask the user how many times the message should be printed. You will need to declare a

variable to store the user's response and use that variable to control the loop. (Remember that all caps is used only for

constants!)

2. Number each line in the output, and add a message at the end of the loop that says how many times the message was

printed. So if the user enters 3, your program should print this:

```
1 I love Computer Science!! 2 I love Computer Science!! 3 I love
Computer Science!! Printed this message 3 times.
```

3. If the message is printed N times, compute and print the sum of the numbers from 1 to N. So for the example above, the

last line would now read:

Printed this message 3 times. The sum of the numbers from 1 to 3 is 6.

Factorials

The `factorial` of n (written $n!$) is the product of the integers between 1 and n . Thus $4! = 1*2*3*4 = 24$. By definition, $0! = 1$. Factorial is not defined for negative numbers.

1. Write a program that asks the user for a non-negative integer and computes and prints the factorial of that integer. You'll need a while loop to do most of the work—this is a lot like computing a sum, but it's a product instead. And you'll need to think about what should happen if the user enters 0.
2. Now modify your program so that it checks to see if the user entered a negative number. If so, the program should print a message saying that a nonnegative number is required and ask the user to enter another number. The program should keep doing this until the user enters a nonnegative number, after which it should compute the factorial of that number.

A Guessing Game

`Guess.java` contains a skeleton for a program to play a guessing game with the user. The program should randomly generate an integer between 1 and 10, then ask the user to try to guess the number. If the user guesses incorrectly, the program should ask them to try again until the guess is correct; when the guess is correct, the program should print a congratulatory message.

1. Using the comments as a guide, complete the program so that it plays the game as described above.
2. Modify the program so that if the guess is wrong, the program says whether it is too high or too low.
3. Now add code to count how many guesses it takes the user to get the number, and print this number at the end with the congratulatory message.
4. Finally, count how many of the guesses are too high and how many are too low. Print these values, along with the total number of guesses, when the user finally guesses correctly.

```
import java.util.Scanner;
import java.util.Random;
public class Guess
```

```
{  
    public static void main(String[] args)  
    {  
        int numToGuess; //Number the user tries to guess  
        int guess; //The user's guess  
        Scanner scan = new Scanner(System.in);  
        Random generator = new Random();  
        //randomly generate the number to guess  
        //print message asking user to enter a guess  
        //read in guess  
        while ( ) //keep going as long as the guess is wrong  
        {  
            //print message saying guess is wrong  
            //get another guess from the user  
        }  
        //print message saying guess is right  
    }  
}
```

Election Day

It's almost election day and the election officials need a program to help tally election results. There are two candidates for office—Polly Tichen and Ernest Orator.

The program's job is to take as input the number of votes each candidate received in each voting precinct and find the total number of votes for each. The program should print out the final tally for each candidate—both the total number of votes each received and the percent of votes each received.

Each iteration of the loop is responsible for reading in the votes from a single precinct and updating the tallies.

A skeleton of the Election.java given below.

1. Add the code to control the loop. You may use either a while loop or a do...while loop. The loop must be controlled by asking the user whether or not there are more precincts to report (that is, more precincts whose votes need to be added in). The user should answer with the character y or n though your program should also allow uppercase responses. The variable response (type String) has already been declared.
2. Add the code to read in the votes for each candidate and find the total votes. Note that variables have already been declared for you to use. Print out the totals and the percentages after the loop.
3. Test your program to make sure it is correctly tallying the votes and finding the percentages AND that the loop control is correct (it goes when it should and stops when it should).
4. The election officials want more information. They want to know how many precincts each candidate carried (won). Add code to compute and print this. You need three new variables: one to count the number of precincts won by Polly, one to count the number won by Ernest, and one to count the number of ties. Test your program after adding this code.

```
import java.util.Scanner;
public class Election
{
    public static void main (String[] args)
    {
        int votesForPolly; // number of votes for Polly in each precinct
        int votesForErnest; // number of votes for Ernest in each precinct
        int totalPolly; // running total of votes for Polly
        int totalErnest; // running total of votes for Ernest
        String response; // answer (y or n) to the "more precincts" question
```

```
Scanner scan = new Scanner(System.in);
System.out.println ();
System.out.println ("Election Day Vote Counting Program");
System.out.println ();
// Initializations
// Loop to "process" the votes in each precinct
// Print out the results
    }
}
```